

# Elucidating the Design Space of Diffusion-Based Generative Models

参考:

hallo128: <https://www.bilibili.com/video/BV1KV5CenEFo>

Double童发发: <https://www.bilibili.com/video/BV1Sn17YHE3B>

董子斌: <https://zhuanlan.zhihu.com/p/682151286>

<https://arxiv.org/abs/2206.00364>

## 0. 概述

这篇论文主要做的事情:

- 设计了一个通用的扩散模型框架, 包括采样方式、训练方式。
- 证明了前面一些比较著名的扩散模型是符合这个框架的, 包括DDPM (VP)、VE、DDIM。这些著名的扩散模型对应的超参数具体取多少。
- 在这个通用的扩散模型框架下, 再设计一个更好的扩散模型EDM。
- 在ODE和SDE下, EDM是如何采样的。

## 1. 各个公式及推导思路

### SDE的原始形式(前向过程)

$$dx = f(x, t) dt + g(t) d\omega_t, \quad (9)$$

原始的SDE形式是在宋扬博士那篇分数模型论文中提出的

《Score-Based Generative Modeling through Stochastic Differential Equations》

### SDE的简化形式(前向过程)

$$dx = f(t) x dt + g(t) d\omega_t. \quad (10)$$

因为VP和VE的 $f(x, t)$ 都是 $f(t)*x$ 的形式, 所以EDM里面做了一点点简化, 得到公式10。

## SDE的扰动核的形式 (EDM前向过程)

$$p_{0t}(\mathbf{x}(t) | \mathbf{x}(0)) = \mathcal{N}(\mathbf{x}(t); s(t) \mathbf{x}(0), s(t)^2 \sigma(t)^2 \mathbf{I}), \quad (11)$$

$$s(t) = \exp\left(\int_0^t f(\xi) d\xi\right), \quad \text{and} \quad \sigma(t) = \sqrt{\int_0^t \frac{g(\xi)^2}{s(\xi)^2} d\xi}. \quad (12)$$

附录B.1: 通过公式10解出来的结果, 用到了《applied stochastic Differential Equations》里面的定理, 得到 $x(t)$ 的期望和方差。

用更具意义的 $s(t)$ 和 $\sigma(t)$ 来替代意义不明的 $f(t)$ 和 $g(t)$

同时, 这个也是SDE的前向过程, 如果 $s(t)$ 和 $\sigma(t)$ 已经定义好了, 只需要一步就可以通过 $x_0$ , 采样得到 $x_t$

## s、 $\sigma$ 、分数函数表示下的ODE的一般形式

$$d\mathbf{x} = \left[ \frac{\dot{s}(t)}{s(t)} \mathbf{x} - s(t)^2 \dot{\sigma}(t) \sigma(t) \nabla_{\mathbf{x}} \log p\left(\frac{\mathbf{x}}{s(t)}; \sigma(t)\right) \right] dt. \quad (4)$$

附录B.2: 通过10、11、12可以把ODE推导出来。

$\dot{s}(t)$ 是 $s(t)$ 对 $t$ 求导

最右边 $\Delta_x \log p(\frac{\mathbf{x}}{s(t)}; \sigma(t))$ 是分数函数, 那个分号“;”表示卷积。分数函数可以就按整体来理解, 因为除了这个部分的其他东西都是比较容易理解的, 只有这个东西很恶心, 我们直接用网络来拟合它。

## 训练目标、以及分数函数

$$\mathbb{E}_{\mathbf{y} \sim p_{\text{data}}} \mathbb{E}_{\mathbf{n} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})} \|D(\mathbf{y} + \mathbf{n}; \sigma) - \mathbf{y}\|_2^2, \text{ then } \nabla_{\mathbf{x}} \log p(\mathbf{x}; \sigma) = (D(\mathbf{x}; \sigma) - \mathbf{x})/\sigma^2, \quad (2, 3)$$

附录B.3证明了, 如果训练目标是公式 (2), 那么分数函数就是公式 (3)

$y$ 是干净的图片;  $n$ 是噪声;  $y+n$ 是加了噪声的图片;  $D$ 是一个神经网络, 输入是加了噪声的图片 和  $\sigma$ , 输出是 预测的干净图片 $y'$

但这个公式3在公式4中的分数函数的形式是不同的, 少了个 $s(t)$ , 附录B.4, 把 $s(t)$ 也引入进来了, 得到:

$$\nabla_{\mathbf{x}} \log p(\mathbf{x}/s(t); \sigma(t)) = \frac{1}{s(t)\sigma(t)^2} (D(\hat{\mathbf{x}}; \sigma(t)) - \hat{\mathbf{x}}). \quad (74)$$

其中,  $\hat{\mathbf{x}} = \mathbf{x}/s(t)$

## s、σ、D表示下的ODE的一般形式

用D来替换ODE里面的分数函数，也就是把公式74代入到公式4中，得到：

$$dx = \left[ \left( \frac{\dot{\sigma}(t)}{\sigma(t)} + \frac{\dot{s}(t)}{s(t)} \right) x - \frac{\dot{\sigma}(t)s(t)}{\sigma(t)} D_{\theta}(x/s(t); \sigma(t)) \right] dt. \quad (80)$$

有了这条式子，当D训练好了，我们就可以做反向的采样了，因为s、σ、D都是已知的，然后按照离散的方式来采样就可以了。

## s、σ、分数函数表示下的SDE的一般形式

<https://www.bilibili.com/video/BV1yYypYRELM>

引入随机性的好处：

This perspective reveals why stochasticity is helpful in practice: The implicit Langevin diffusion drives the sample towards the desired marginal distribution at a given time, actively correcting for any errors made in earlier sampling steps. On the other hand, approximating the Langevin term with discrete SDE solver steps introduces error in itself. Previous results [3, 24, 47, 49] suggest that

$$dx_{\pm} = \underbrace{-\dot{\sigma}(t)\sigma(t)\nabla_x \log p(x; \sigma(t)) dt}_{\text{probability flow ODE (Eq. 1)}} \pm \underbrace{\beta(t)\sigma(t)^2 \nabla_x \log p(x; \sigma(t)) dt + \sqrt{2\beta(t)}\sigma(t) d\omega_t}_{\text{Langevin diffusion SDE}}, \quad (6)$$

其中， $\beta(t) = \dot{\sigma}(t)/\sigma(t)$ ，分数函数可以用公式74替代

±号：+号表示正向；-号表示逆向。我们做采样，只需要逆向即可。所以其实可以进一步化简为：

$$dx_- = -2\dot{\sigma}(t)\sigma(t)\Delta_x \log p(x; \sigma(t))dt + \sqrt{2\sigma(t)\dot{\sigma}(t)}d\omega_t$$

这里EDM默认s(t)=1，所以这里看不到s(t)

## 2. 通用训练

根据[公式80](#)

网络是D，输入是加了噪声的图x，以及噪声的方差σ

D已经可以用来做ODE采样了，但还有点问题，训练它的时候不太好训练：

- 如果网络改成预测噪声，那么就更容易训练一点。（DDPM做过实验）
- 因为D输入的x的取值区间的范围可能是很大的（因为t比较小的时候，x比较接近y；而t比较大的时候，x和y差很远），网络如果需要针对min和max都有很好的结果，可能比较困难。
- 同样的因为D的预测改为了预测噪声，输出的噪声范围可能也是很大的，预测也有一定的困难。

那就设计一个方式，把D的输入设定在一个比较小的范围，输出也设定在一个比较小的范围。这个可以通过设计一些缩放因子来实现，比如只考虑输出的情况下，做这样的设计：

$$D_{\theta}(\mathbf{x}; \sigma) = \mathbf{x} - \sigma F_{\theta}(\cdot)$$

$F_{\theta}$ 是一个新的网络，预测是一个比较小的范围的噪声，预测完后，乘上噪声 $\sigma$ ，可以让这个小的范围的噪声的范围变得更大。

进一步推广，可以把每个地方都加这样的倍率进行缩放：

$$D_{\theta}(\mathbf{x}; \sigma) = c_{\text{skip}}(\sigma) \mathbf{x} + c_{\text{out}}(\sigma) F_{\theta}(c_{\text{in}}(\sigma) \mathbf{x}; c_{\text{noise}}(\sigma)), \quad (7)$$

在这种情况下，训练的损失函数变成了这样：

$$\mathbb{E}_{\sigma, \mathbf{y}, \mathbf{n}} \left[ \underbrace{\lambda(\sigma) c_{\text{out}}(\sigma)^2}_{\text{effective weight}} \left\| \underbrace{F_{\theta}(c_{\text{in}}(\sigma) \cdot (\mathbf{y} + \mathbf{n}); c_{\text{noise}}(\sigma))}_{\text{network output}} - \underbrace{\frac{1}{c_{\text{out}}(\sigma)} (\mathbf{y} - c_{\text{skip}}(\sigma) \cdot (\mathbf{y} + \mathbf{n}))}_{\text{effective training target}} \right\|_2^2 \right]. \quad (8)$$

### 3. Table1: VP、VE、DDIM、EDM的具体超参

框架已经基本定义完了，那么在这个框架下，之前比较著名的扩散模型对应的超参就是table1里面列出来的样子。具体的证明可以参考附录

Table 1: Specific design choices employed by different model families.  $N$  is the number of ODE solver iterations that we wish to execute during sampling. The corresponding sequence of time steps is  $\{t_0, t_1, \dots, t_N\}$ , where  $t_N = 0$ . If the model was originally trained for specific choices of  $N$  and  $\{t_i\}$ , the originals are denoted by  $M$  and  $\{u_j\}$ , respectively. The denoiser is defined as  $D_{\theta}(\mathbf{x}; \sigma) = c_{\text{skip}}(\sigma) \mathbf{x} + c_{\text{out}}(\sigma) F_{\theta}(c_{\text{in}}(\sigma) \mathbf{x}; c_{\text{noise}}(\sigma))$ ;  $F_{\theta}$  represents the raw neural network layers.

|                                                | VP [49]                                               | VE [49]                                                                | iDDPM [37] + DDIM [47]                                                                  | Ours (“EDM”)                                                                                                               |
|------------------------------------------------|-------------------------------------------------------|------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>Sampling (Section 3)</b>                    |                                                       |                                                                        |                                                                                         |                                                                                                                            |
| ODE solver                                     | Euler                                                 | Euler                                                                  | Euler                                                                                   | 2 <sup>nd</sup> order Heun                                                                                                 |
| Time steps                                     | $t_{i < N}$                                           | $1 + \frac{i}{N-1}(\epsilon_s - 1)$                                    | $\sigma_{\max}^2 \left(\sigma_{\min}^2 / \sigma_{\max}^2\right)^{\frac{i}{N-1}}$        | $u_{\lfloor j_0 + \frac{M-1-j_0}{N-1} i + \frac{1}{2} \rfloor}$ , where                                                    |
|                                                |                                                       |                                                                        | $u_M = 0$                                                                               | $(\sigma_{\max}^{\frac{1}{\rho}} + \frac{i}{N-1}(\sigma_{\min}^{\frac{1}{\rho}} - \sigma_{\max}^{\frac{1}{\rho}}))^{\rho}$ |
|                                                |                                                       |                                                                        | $u_{j-1} = \sqrt{\frac{u_j^2 + 1}{\max(\bar{\alpha}_{j-1} / \bar{\alpha}_j, C_1)}} - 1$ |                                                                                                                            |
| Schedule                                       | $\sigma(t)$                                           | $\sqrt{e^{\frac{1}{2}\beta_d t^2 + \beta_{\min} t} - 1}$               | $\sqrt{t}$                                                                              | $t$                                                                                                                        |
| Scaling                                        | $s(t)$                                                | $1 / \sqrt{e^{\frac{1}{2}\beta_d t^2 + \beta_{\min} t}}$               | 1                                                                                       | 1                                                                                                                          |
| <b>Network and preconditioning (Section 5)</b> |                                                       |                                                                        |                                                                                         |                                                                                                                            |
| Architecture of $F_{\theta}$                   | DDPM++                                                | NCSN++                                                                 | DDPM                                                                                    | (any)                                                                                                                      |
| Skip scaling                                   | $c_{\text{skip}}(\sigma)$                             | 1                                                                      | 1                                                                                       | $\sigma_{\text{data}}^2 / (\sigma^2 + \sigma_{\text{data}}^2)$                                                             |
| Output scaling                                 | $c_{\text{out}}(\sigma)$                              | $-\sigma$                                                              | $\sigma$                                                                                | $-\sigma$                                                                                                                  |
| Input scaling                                  | $c_{\text{in}}(\sigma)$                               | $1 / \sqrt{\sigma^2 + 1}$                                              | $1 / \sqrt{\sigma^2 + 1}$                                                               | $\sigma \cdot \sigma_{\text{data}} / \sqrt{\sigma_{\text{data}}^2 + \sigma^2}$                                             |
| Noise cond.                                    | $c_{\text{noise}}(\sigma)$                            | $(M - 1) \sigma^{-1}(\sigma)$                                          | $\ln(\frac{1}{2}\sigma)$                                                                | $1 / \sqrt{\sigma^2 + \sigma_{\text{data}}^2}$                                                                             |
|                                                |                                                       |                                                                        | $M - 1 - \arg \min_j  u_j - \sigma $                                                    | $\frac{1}{4} \ln(\sigma)$                                                                                                  |
| <b>Training (Section 5)</b>                    |                                                       |                                                                        |                                                                                         |                                                                                                                            |
| Noise distribution                             | $\sigma^{-1}(\sigma) \sim \mathcal{U}(\epsilon_t, 1)$ | $\ln(\sigma) \sim \mathcal{U}(\ln(\sigma_{\min}), \ln(\sigma_{\max}))$ | $\sigma = u_j, \quad j \sim \mathcal{U}\{0, M - 1\}$                                    | $\ln(\sigma) \sim \mathcal{N}(P_{\text{mean}}, P_{\text{std}}^2)$                                                          |
| Loss weighting                                 | $\lambda(\sigma)$                                     | $1 / \sigma^2$                                                         | $1 / \sigma^2$ (note: *)                                                                | $(\sigma^2 + \sigma_{\text{data}}^2) / (\sigma \cdot \sigma_{\text{data}})^2$                                              |
| <b>Parameters</b>                              |                                                       |                                                                        |                                                                                         |                                                                                                                            |
|                                                | $\beta_d = 19.9, \beta_{\min} = 0.1$                  | $\sigma_{\min} = 0.02$                                                 | $\bar{\alpha}_j = \sin^2(\frac{\pi}{2} \frac{j}{M(C_2 + 1)})$                           | $\sigma_{\min} = 0.002, \sigma_{\max} = 80$                                                                                |
|                                                | $\epsilon_s = 10^{-3}, \epsilon_t = 10^{-5}$          | $\sigma_{\max} = 100$                                                  | $C_1 = 0.001, C_2 = 0.008$                                                              | $\sigma_{\text{data}} = 0.5, \rho = 7$                                                                                     |
|                                                | $M = 1000$                                            |                                                                        | $M = 1000, j_0 = 8^{\dagger}$                                                           | $P_{\text{mean}} = -1.2, P_{\text{std}} = 1.2$                                                                             |

\* iDDPM also employs a second loss term  $L_{\text{vib}}$

<sup>†</sup> In our tests,  $j_0 = 8$  yielded better FID than  $j_0 = 0$  used by iDDPM

## EDM的设计理念

- 理念一：EDM 希望神经网络的输入有稳定的标准差 1
- 理念二：EDM 希望神经网络的拟合目标有稳定的标准差 1
- 理念三：EDM 希望 loss 对不同的噪声标准差分配的权重是均等的

## 4. EDM的ODE采样（反向过程）

**Algorithm 1** Deterministic sampling using Heun's 2<sup>nd</sup> order method with arbitrary  $\sigma(t)$  and  $s(t)$ .

```
1: procedure HEUN_SAMPLER( $D_\theta(\mathbf{x}; \sigma)$ ,  $\sigma(t)$ ,  $s(t)$ ,  $t_i \in \{0, \dots, N\}$ )
2:   sample  $\mathbf{x}_0 \sim \mathcal{N}(\mathbf{0}, \sigma^2(t_0) s^2(t_0) \mathbf{I})$  ▷ Generate initial sample at  $t_0$ 
3:   for  $i \in \{0, \dots, N-1\}$  do ▷ Solve Eq. 4 over  $N$  time steps
4:      $\mathbf{d}_i \leftarrow \left( \frac{\dot{\sigma}(t_i)}{\sigma(t_i)} + \frac{\dot{s}(t_i)}{s(t_i)} \right) \mathbf{x}_i - \frac{\dot{\sigma}(t_i) s(t_i)}{\sigma(t_i)} D_\theta \left( \frac{\mathbf{x}_i}{s(t_i)}; \sigma(t_i) \right)$  ▷ Evaluate  $d\mathbf{x}/dt$  at  $t_i$ 
5:      $\mathbf{x}_{i+1} \leftarrow \mathbf{x}_i + (t_{i+1} - t_i) \mathbf{d}_i$  ▷ Take Euler step from  $t_i$  to  $t_{i+1}$ 
6:     if  $\sigma(t_{i+1}) \neq 0$  then ▷ Apply 2nd order correction unless  $\sigma$  goes to zero
7:        $\mathbf{d}'_i \leftarrow \left( \frac{\dot{\sigma}(t_{i+1})}{\sigma(t_{i+1})} + \frac{\dot{s}(t_{i+1})}{s(t_{i+1})} \right) \mathbf{x}_{i+1} - \frac{\dot{\sigma}(t_{i+1}) s(t_{i+1})}{\sigma(t_{i+1})} D_\theta \left( \frac{\mathbf{x}_{i+1}}{s(t_{i+1})}; \sigma(t_{i+1}) \right)$  ▷ Eval.  $d\mathbf{x}/dt$  at  $t_{i+1}$ 
8:        $\mathbf{x}_{i+1} \leftarrow \mathbf{x}_i + (t_{i+1} - t_i) \left( \frac{1}{2} \mathbf{d}_i + \frac{1}{2} \mathbf{d}'_i \right)$  ▷ Explicit trapezoidal rule at  $t_{i+1}$ 
9:   return  $\mathbf{x}_N$  ▷ Return noise-free sample at  $t_N$ 
```

注意：它这里 $x_0$ 是那张纯噪声的图。 $x_N$ 就是没有噪声的图

heun的主要思想：

欧拉采样（一阶）：在当前时刻 $t_i$ 时，可以通过ODE的式子得到 $dx_i$ ，然后得到

$$x_{i+1} = x_i + (t_{i+1} - t_i) dx_i$$

heun采样（二阶）：欧拉估计出 $x_{i+1}$ 之后，又可以估计出 $dx_{i+1}$ ，然后再进一步更新

$$x_{i+1} = x_i + (t_{i+1} - t_i) (dx_i + dx_{i+1}) * 0.5$$

### Trapezoidal rule

A simple Reimann sum approximates an integral with one value:

对应欧拉采样

$$\int_a^b f(x) dx \approx f(a)(b-a)$$

The value of the function, however, changes across  $[a, b]$ ,  
so the trapezoidal rule averages the two end-points:

对应heun采样

$$\int_a^b f(x) dx \approx \frac{f(a) + f(b)}{2} (b-a)$$

eula相当于矩形策略；heun相当于梯形策略。

## 5. EDM的SDE采样（反向过程）

<https://www.bilibili.com/video/BV1Sn17YHE3B>

如果按照SDE的公式+EDM随机采样的前提假设 ( $\sigma(t) = t$ )，那么SDE可以进一步化简成：

$$dx_- = -2\dot{\sigma}(t)\sigma(t)\Delta_x \log p(x; \sigma(t))dt + \sqrt{2\sigma(t)\dot{\sigma}(t)}d\omega_t$$

$$dx_- = -2(D - x)/t * dt + \sqrt{2t}d\omega_t$$

EDM的随机性采样没有完全依赖于SDE，而是做了一些类似SDE的操作，引入了随机性。（感觉如果对维纳函数做一些特殊的设定，可能也可以变成下面采样的样子）

随机性采样：

### 随机性采样

随机性采样并没有完全依赖SDE，而是在确定性采样的基础上，先给原有样本 $x_i$ 进一步加噪，有一个回退的操作，因此图像生成效果通常会比ODE效果好。

**Algorithm 2** Our stochastic sampler with  $\sigma(t) = t$  and  $s(t) = 1$ .

```
1: procedure STOCHASTICSAMPLER( $D_\theta(x; \sigma)$ ,  $t_i \in \{0, \dots, N\}$ ,  $\gamma_i \in \{0, \dots, N-1\}$ ,  $S_{\text{noise}}$ )
2:   sample  $x_0 \sim \mathcal{N}(0, t_0^2 \mathbf{I})$ 
3:   for  $i \in \{0, \dots, N-1\}$  do
4:     sample  $\epsilon_i \sim \mathcal{N}(0, S_{\text{noise}}^2 \mathbf{I})$ 
5:      $\hat{t}_i \leftarrow t_i + \gamma_i t_i$ 
6:      $\hat{x}_i \leftarrow x_i + \sqrt{\hat{t}_i^2 - t_i^2} \epsilon_i$ 
7:      $d_i \leftarrow (\hat{x}_i - D_\theta(\hat{x}_i; \hat{t}_i)) / \hat{t}_i$ 
8:      $x_{i+1} \leftarrow \hat{x}_i + (t_{i+1} - \hat{t}_i) d_i$ 
9:     if  $t_{i+1} \neq 0$  then
10:       $d'_i \leftarrow (x_{i+1} - D_\theta(x_{i+1}; t_{i+1})) / t_{i+1}$ 
11:       $x_{i+1} \leftarrow \hat{x}_i + (t_{i+1} - \hat{t}_i) (\frac{1}{2} d_i + \frac{1}{2} d'_i)$ 
12:   return  $x_N$ 
```

$\triangleright \gamma_i = \begin{cases} \min\left(\frac{S_{\text{churn}}}{N}, \sqrt{2}-1\right) & \text{if } t_i \in [S_{\text{tmin}}, S_{\text{tmax}}] \\ 0 & \text{otherwise} \end{cases}$

$\triangleright$  Select temporarily increased noise level  $\hat{t}_i$   
 $\triangleright$  Add new noise to move from  $t_i$  to  $\hat{t}_i$   
 $\triangleright$  Evaluate  $dx/dt$  at  $\hat{t}_i$   
 $\triangleright$  Take Euler step from  $\hat{t}_i$  to  $t_{i+1}$   
 $\triangleright$  Apply 2<sup>nd</sup> order correction

视频里面讲的比较详细。 $\gamma$ 的设定刚好让 $x_i$ 的回退程度不会超过 $x_{i-1}$ 的噪声程度。

$S_{\text{noise}}$ 是一个超参。

把 $\sigma(t) = t$ 和 $s(t)=1$ 代入到公式6，可以化简得到第7行的采样结果

```
# comfyui: comfy/k_diffusion/sampling.py

def sample_heun(model, x, sigmas, extra_args=None, callback=None, disable=None,
s_churn=0., s_tmin=0., s_tmax=float('inf'), s_noise=1.):
    """Implements Algorithm 2 (Heun steps) from Karras et al. (2022)."""
    extra_args = {} if extra_args is None else extra_args
    s_in = x.new_ones([x.shape[0]])
    for i in trange(len(sigmas) - 1, disable=disable):
        if s_churn > 0:
            gamma = min(s_churn / (len(sigmas) - 1), 2 ** 0.5 - 1) if s_tmin <=
sigmas[i] <= s_tmax else 0.
            sigma_hat = sigmas[i] * (gamma + 1)
        else:
            gamma = 0
```

B站: <https://space.bilibili.com/389001702>

转载请注明出处

github: <https://github.com/joe-zxh/ComfyUI>

```

        sigma_hat = sigmas[i]

    sigma_hat = sigmas[i] * (gamma + 1)
    if gamma > 0:
        eps = torch.randn_like(x) * s_noise
        x = x + eps * (sigma_hat ** 2 - sigmas[i] ** 2) ** 0.5
    denoised = model(x, sigma_hat * s_in, **extra_args)
    d = to_d(x, sigma_hat, denoised) # dx / dt
    if callback is not None:
        callback({'x': x, 'i': i, 'sigma': sigmas[i], 'sigma_hat':
sigma_hat, 'denoised': denoised})
    dt = sigmas[i + 1] - sigma_hat
    if sigmas[i + 1] == 0:
        # Euler method
        x = x + d * dt
    else:
        # Heun's method
        x_2 = x + d * dt
        denoised_2 = model(x_2, sigmas[i + 1] * s_in, **extra_args)
        d_2 = to_d(x_2, sigmas[i + 1], denoised_2)
        d_prime = (d + d_2) / 2
        x = x + d_prime * dt
    return x

```